Aaron N. Cutshall, DHA, MSHI

Silent But Deadly – Hidden SQL Performance Assassins



SQLRV, LLC
HEALTHCARE INFORMATICS
https://www.sqlrv.com



Agenda

- Who is responsible for database performance?
- The Major Performance Assassins:
  - Non-SARGable Queries
  - Data Bloat
  - Conditional Branching
  - Recursive CTEs
  - Hidden Loops
- What this all means for you

Copyright © 2026, SQLRV, LLC - All Rights Reserved



Something to consider…



"Make it work, make it right, make it fast."

– Kent Beck *(Software engineer and creator of* Extreme Programming*)*

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Who is responsible for database performance?

- Short Answer: You
- Role doesn’t matter
  - Data Architect
  - Database Developer/Engineer
  - Database Administrator
- Bad design, bad code, or poor maintenance all contribute
- Performance improvement is a team effort



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Who is responsible for database performance?

- Brent Ozar: "80% of your performance issues are caused by 20% of your queries.“
  - Higher logic complexity – fewer people who understand it
  - Larger query code – more code, more things to go wrong
  - Accounts for the majority of index requirements
- Data Architect is focused on before & after – not during
- DBA is focused on the machine state – not just the code
- Data Developer must be responsible for code written
- It is NOT to be pushed out to production support!



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Hidden Performance Assassins



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Sergeant Non-SARGable



- A query is SARGable (Search Argument Able) if the engine can use an index seek to find matching rows.
- Non-SARGable queries use index or table scans (RBAR – Row by Agonizing Row)
- Major causes of Non-SARGable queries:
  - Functions on filtering columns
  - Implicit data conversions w/ mismatched data types
  - Leading Wildcards in LIKE operations

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Sergeant Non-SARGable – Functions on Filters

- Using functions on filters prevents index usage
  - Example: `SELECT * FROM Orders WHERE YEAR(OrderDate) = 2026;`
  - Even with an index on OrderDate, the database cannot seek that index.
  - The YEAR() function is evaluated for every single row in the table
  - Result: Costly Index Scan or Table Scan instead of an efficient Index Seek.
- Correct Approach:
  - Rewrite the query so the column remains bare

```
SELECT * FROM Orders WHERE OrderDate BETWEEN CAST('2026-01-01' AS
  DATE) AND CAST('2026-12-31' AS DATE);
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Sergeant Non-SARGable – Functions on Filters

- Causes High CPU and I/O Overhead
  - Without index seeks, SQL Server reads pages sequentially
  - Data moves from disk into memory (I/O)
  - Function applied to every row (CPU)
  - On large tables (millions of rows) it causes
    - High CPU spikes
    - Heavy disk activity
    - General query slowdowns

Copyright © 2026, SQLRV, LLC - All Rights Reserved



Sergeant Non-SARGable – Functions on Filters

- Inaccurate Cardinality Estimates (Bad Query Plans)
  - Query optimizer relies on statistics for row estimates
  - When using a function (e.g., `UPPER(LastName) = 'SMITH'`), statistics on LastName do not match the expression
  - Optimizer often guesses an inaccurate rows count (cardinality)
  - Results in a poor execution plan—such as a nested loop join where a hash match would be far faster.

Copyright © 2026, SQLRV, LLC - All Rights Reserved



Sergeant Non-SARGable – Data Type Mis-Matches

- Remember:
  - int <> bigint
  - bigint <> number(6,2)
  - varchar <> int
  - char <> varchar <> text
  - varchar(16) <> varchar(8)
  - varchar <> nvarchar
- Implicit conversions:
  - Hidden operations
  - May have unexpected side effects
- Use explicit data conversions
  - CAST() functions as needed
  - Example: `WHERE @MyDate = CAST('2025-03-05' AS DATE);`



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Sergeant Non-SARGable – Data Type Mis-Matches

- Has unintended side-effects
  - Unexpected conversions with consequences
  - Data export formatting issues
- Parameter values
  - When called from external code, not all datatypes are distinct
  - Stored procedure parameters need to match the data as used
  - Examples:
    - String (char, varchar, text)
    - Numeric (boolean, shortint, int, bigint, real, float, decimal/numeric)
    - Date/Time (date, time, timestamp, interval, with/without time zone)



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Sergeant Non-SARGable – Data Type Mis-Matches

- Constant Declarations
  - Example: Using a string instead of numeric: WHERE MyIntVal = ‘1’
- Table data types not matching
  - Permanent table or temp table
  - INSERT or UPDATE
- Implicit conversion side effect example:

```
SELECT CONCAT(1, 2, 3) + 4
```

  - Result:

```
SELECT CONCAT('1', '2', '3') + 4
SELECT '123' + 4
SELECT 123 + 4
SELECT 127
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Sergeant Non-SARGable – Wildcard Searches

- Leading wildcard searches (`WHERE LastName LIKE '%smith'`)
  - A common causes of database query degradation
  - Invalidates B-Tree index seeks
  - Forces index scans or worse, table scans
  - The optimizer has no starting anchor, so every row must be inspected
  - Increases CPU, Memory, and Disk time consumption
  - Does not allow for accurate cardinality estimates
  - May cause inappropriate join operations



Copyright © 2026, SQLRV, LLC - All Rights Reserved

The Data Blob



- Over-fetching too many columns or rows
  - Ties up memory and network
  - Costs more resources than necessary
  - Increases intermediate row generation
  - Can increase latency with RAM spilled to disk
  - Prevents the use of covering indexes
- Major causes of data bloat:
  - SELECT * or other unnecessary columns
  - Incorrect joins or insufficient filtering criteria
  - Deep OFFSET pagination
  - Insufficient filtering before passing to application
  - Multiply nested views

Copyright © 2026, SQLRV, LLC - All Rights Reserved

The Data Blob

- SELECT * or other unnecessary columns
  - Pulls columns in to working memory, typing up resources
  - Hides the columns that are available
  - May also include unexpected columns that were added
- Joins with insufficient filtering criteria
  - Query optimizers have improved, but still can’t read minds
  - Insufficient criteria can affect cardinality estimates
    - Affects the final join algorithm selection
    - Could cause execution time to be hours instead of milliseconds
  - Could short-circuit the optimizer’s ability to reduce joins



Copyright © 2026, SQLRV, LLC - All Rights Reserved

The Data Blob

- Inappropriate use of an OUTER JOIN can increase intermediate row counts
  - Use of a LEFT JOIN instead of an INNER JOIN then put filter in WHERE clause:

```
SELECT  *
FROM    some_table t
LEFT JOIN some_other_table o
    ON  o.ID = t.FK_ID
WHERE   o.code = 'ABC123';
```

  - Turns a LEFT JOIN into a *very expensive* INNER JOIN!
  - Keep queries up to date with table and code changes



Copyright © 2026, SQLRV, LLC - All Rights Reserved

The Data Blob

- Deep OFFSET pagination (LIMIT 50 OFFSET 500000)
  - Usually used to paginate record sets for display purposes
  - Each query execution starts over, and results are NOT cached (especially in serverless or stateless mode)
  - Sorted sets (common) must fetch all rows – again
- Insufficient filtering before passing to application
  - Pulling massive data set only to have the app filter it
  - Uses resources unnecessarily and takes extra time



Copyright © 2026, SQLRV, LLC - All Rights Reserved

The Data Blob

- Multiply nested views
  - ViewX calls ViewY which in turn calls ViewZ
  - Optimizer cannot traverse down multiple calls
  - ViewZ has computed aggregate columns used for ViewY
  - ViewX selects only 3 columns from ViewY (not the computed columns)
  - Computed columns are still executed for each row generated for ViewX



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Scorched Cursor

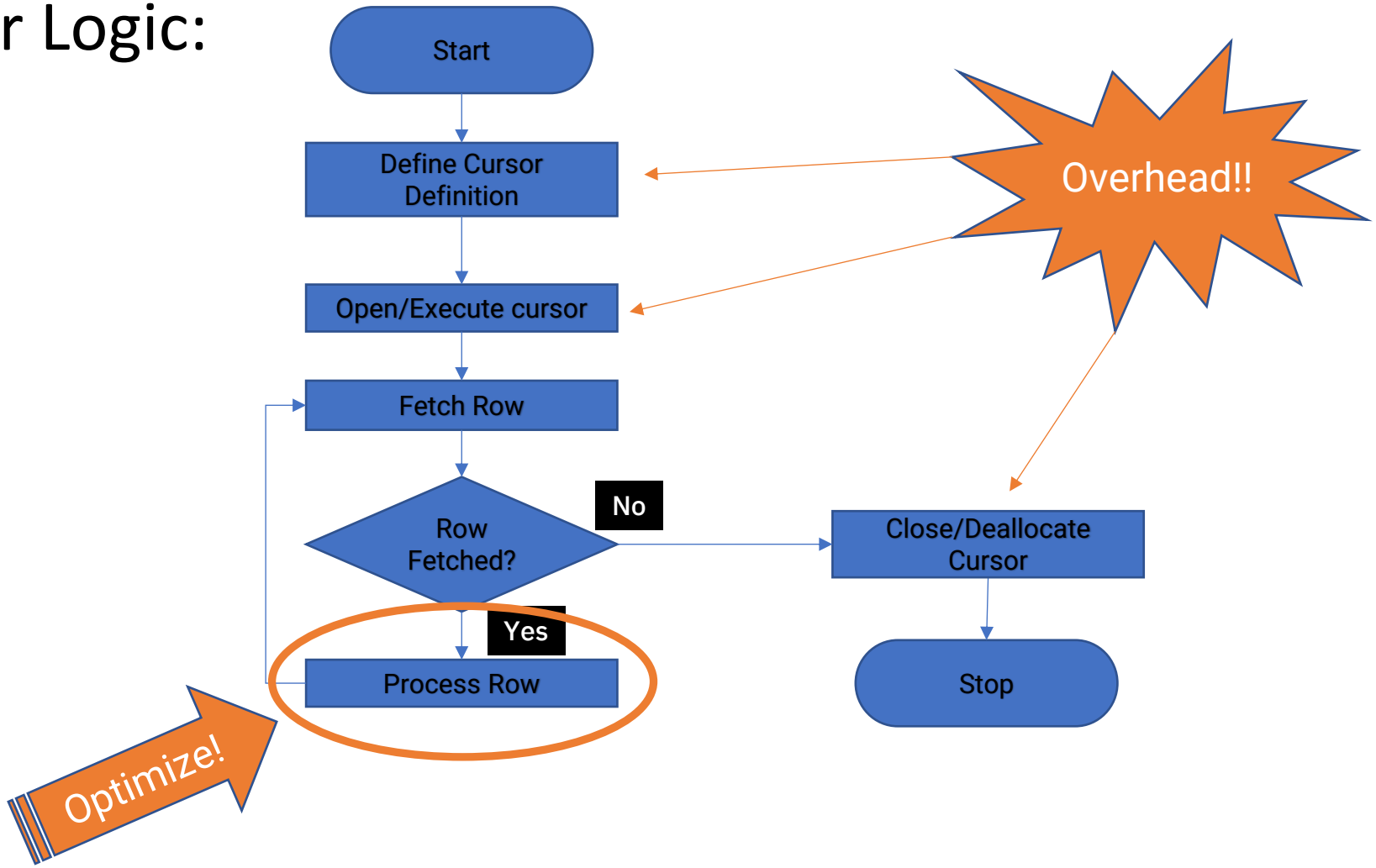


- Heavy reliance on cursors
  - Causes massive overhead using default settings
  - Most often can be replaced by a lesser evil: a WHILE loop
- Major impacts by cursors:
  - Cursors are the embodiment of RBAR processing!
  - To use, a cursor requires a type of loop
  - Favored by procedural programmers
  - Expensive on resources (memory, I/O, etc.)
  - Default cursor definition allows for:
    - Read/write operations
    - Forward/backward motion

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Scorched Cursor

- Cursor Logic:



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Scorched Cursor

- Cursors are the embodiment of RBAR processing!
- To use, a cursor requires a type of loop
- Favored by procedural programmers
- Expensive on resources (memory, I/O, etc.)
- Default cursor definition allows for:
  - Read/write operations
  - Forward/backward motion



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Scorched Cursor

- Use the query from the cursor to populate temp table
  - Add identity column for sequence
  - Insert data using cursor query
  - Use a variable for each sequence value & max sequence
  - Within FOR loop, read and process each record
- Reduces overhead involved in managing a cursor
  - Internally cursors populate a temp table
  - Ensures a NO SCROLL (forward only) movement



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Scorched Cursor

- Cursor Scenario: Generate certificates for recently completed employee training

```
DECLARE @emp_train_id BIGINT, -- employee training id
        @emp_name VARCHAR(256), -- employee name
        @course_desc VARCHAR(256), -- course description
        @completed_dt DATE; -- date course completed

DECLARE db_cursor CURSOR FOR
    SELECT  et.Employee_Training_ID, CONCAT(e.First_Name, ' ', e.Last_Name), t.Course_Desc, et.Course_Completed_DT
    FROM    HR.Employee_Training et
    INNER JOIN HR.Employee e ON e.Employee_ID = et.Employee_ID
    INNER JOIN HR.Training t ON t.Course_ID = et.Course_ID
    WHERE   et.Course_Completed_DT IS NOT NULL AND et.Certificate_Generated_DT IS NULL;

OPEN db_cursor;
FETCH NEXT FROM db_cursor INTO @emp_train_id, @emp_name, @course_desc, @completed_dt;
WHILE @@FETCH_STATUS = 0 BEGIN
    EXEC HR.sp_Generate_Certificate(@emp_name, @course_desc, @completed_dt);

    UPDATE  HR.Employee_Training
        SET Certificated_Generated_DT = GETDATE()
    WHERE   Employee_Training_ID = @emp_train_id;

    FETCH NEXT FROM db_cursor INTO @emp_train_id, @emp_name, @course_desc, @completed_dt;
END

CLOSE db_cursor;
DEALLOCATE db_cursor;
```

Copyright © 2026, SQLRV, LLC - All Rights Reserved



Scorched Cursor

- **Replacement:** Generate certificates for recently completed employee training

```
DECLARE @emp_train_id BIGINT, -- employee training id
        @emp_name VARCHAR(256), -- employee name
        @course_desc VARCHAR(256), -- course description
        @completed_dt DATE; -- date course completed

--Declare a memory table
DECLARE @Cert_Table TABLE (
    seq_nbr INT IDENTITY(1,1) NOT NULL, --THE IDENTITY STATEMENT IS IMPORTANT!
    Employee_Training_ID BIGINT,
    Employee_Full_Name VARCHAR(256),
    Course_Desc VARCHAR(256),
    Completed_DT DATE
    );

--Populate this table with the required item category values using the same query as for the cursor
INSERT INTO @Cert_Table (Employee_Training_ID, Employee_Full_Name, Course_Description, Completed_Date)
SELECT  et.Employee_Training_ID, CONCAT(e.First_Name, ' ', e.Last_Name), t.Course_Desc, et.Course_Completed_DT
FROM    HR.Employee_Training et
INNER JOIN HR.Employee e ON e.Employee_ID = et.Employee_ID
INNER JOIN HR.Training t ON t.Course_ID = et.Course_ID
WHERE   et.Course_Completed_DT IS NOT NULL AND et.Certificate_Generated_DT IS NULL;

SET @i = @@ROWCOUNT;
WHILE @i > 0 BEGIN
    SELECT @emp_train_id = Employee_Training_ID,
           @emp_name = Employee_Full_Name,
           @course_desc = Course_Desc,
           @completed_dt = Completed_DT
    FROM   @Cert_Table
    WHERE  seq_nbr = @i;

    EXEC HR.sp_Generate_Certificate @emp_name, @course_desc, @completed_dt;

    UPDATE HR.Employee_Training
       SET Certificated_Generated_DT = GETDATE()
    WHERE  Employee_Training_ID = @emp_train_id;

    SET @i -= 1;
END
```

Copyright © 2026, SQLRV, LLC - All Rights Reserved



The Loopers

- Hidden RBAR Loops
  - Loops behind the scenes
  - Deceptive: Looks set-based
- Major causes of Hidden Loops:
  - Query variable assignment
  - Non-inlineable User Defined Scalar functions
  - Correlated sub-queries
  - Recursive CTEs
- Alternatives
  - Inline Table-Value Functions
  - The Tally Table/Function



Copyright © 2026, SQLRV, LLC - All Rights Reserved

The Loopers – Query variable assignment

- String concatenation from a query with a variable
  - Single assignment will get the last value returned

```
select some_variable = t.some_field
from   some_table t
where  t.multiple_rows_returned = true;
```

  - Select and reassignment processed for each row

```
select my_text_var = coalesce(my_text_var || '|', '') + t.some_text_field
from   some_table t
where  t.multiple_rows_returned = true;
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

The Loopers – Scalar Functions

- Hidden Loops – Function called for each row
- Especially Multi-Statement Scalar Function
- Scalar Function Calls

```
select udf_My_Complex_Scalar_Function(t.some_value)
from   some_table t
where  t.multiple_rows_returned = true;
```

- Replace with
  - Inlineable Scalar Function
  - Inlineable Table-Valued Function
- SQL Server rules for inlining



Copyright © 2026, SQLRV, LLC - All Rights Reserved

The Deliberate Loop

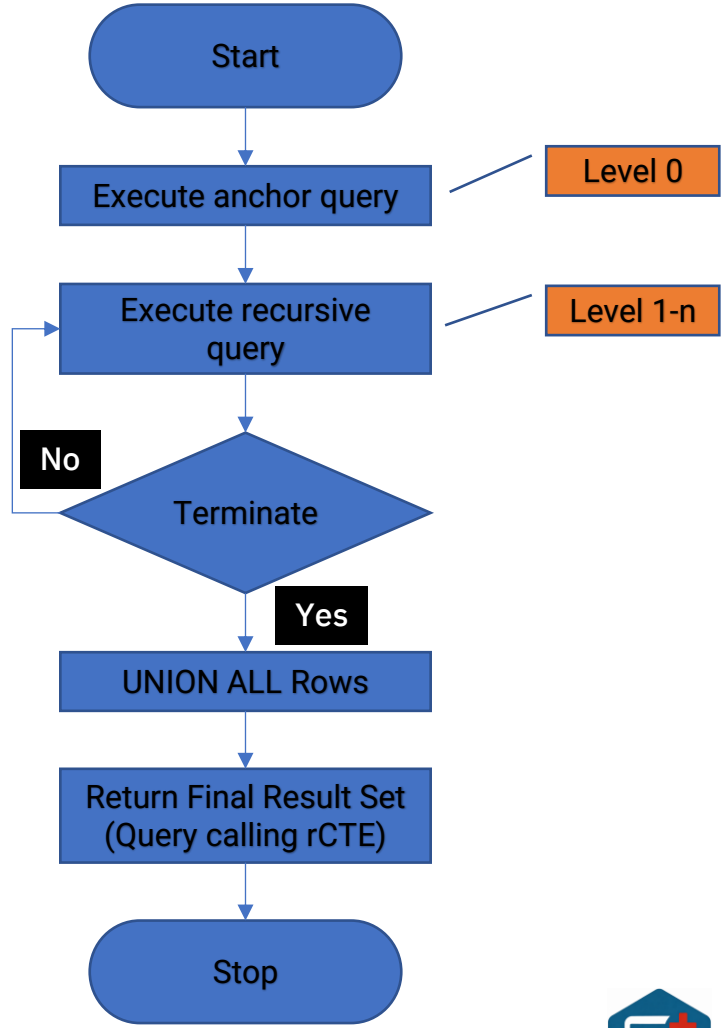
- Loops *MAY* be required if:
  - Performing an operation that is not set-based
    - Execute a stored procedure
    - Perform external process
  - Execution order is critical
  - Necessary to break up operations
    - Better manage system resources
    - Replace a more expensive operation (ex. Cursor or Recursive CTE)



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Recursive CTE

- Common Table Expression that calls itself
  - Is a hidden loop through each level of recursion
  - Requires 3x (or more) the resources of a simple while-loop
  - Each loop requires the results of the previous loop
  - Creates temporary tables behind the scenes
  - Can lock up a server if max levels of recursion are ignored
  - Hierarchy does not eliminate the need for recursion



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Tally



- 
- Sequence number to track things
- Used in queries to track:
  - Characters in a string
  - Rows in a table
  - Pretty much anything you can imagine
- Demo!!

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Sub-Query

- Sub-queries can majorly impact the overall query
  - They must execute first to provide results for joining
  - All results are generated, then filtered by join criteria
- Major impacts by sub-queries:
  - If poorly written, they can turn the overall query into an RBAR loop
  - No indexes are possible forcing full query result scans
  - They may pull more data than required by the join criteria
  - Depending on the execution plan, they may unexpectedly use internal temporary tables
- Solutions
  - For large result sets, temp tables with indexes may be better
  - May need to break large queries with many sub-queries into separate queries



Copyright © 2026, SQLRV, LLC - All Rights Reserved

The Assassin's UNION

- Improper usage of UNION
  - Hidden SORT & DISTINCT operations
  - Often used unnecessarily
- Major impacts by bad UNIONs:
  - Must wait for all queries to complete
  - Not all queries execute in parallel
  - All query results are merged & sorted
  - A DISTINCT operation is performed
- Alternative: UNION ALL



Copyright © 2026, SQLRV, LLC - All Rights Reserved

UNION ALL



- Eliminates the SORT & DISTINCT operations
- Best when each query is already distinct or duplicates will be filtered later
- Use an outer query with a ROW_NUMBER window function
- Avoid conditional procedural branching
- Replace inefficient OR statements

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Conditional Branching

- Typical in procedural processing
  - Not inherent in SQL operations
  - T-SQL procedural abilities extend SQL
  - Could encourage RBAR processing

```
IF @Todays_Color = 'Blue’
    SELECT MyFirstColumn, MySecondColumn
    FROM   Blue_Table;
ELSE
    SELECT MyFirstColumn, MySecondColumn
    FROM   Red_Table;
```

- Instead, use UNION ALL:

```
SELECT MyFirstColumn, MySecondColumn
FROM   Blue_Table
WHERE  @Todays_Color = 'Blue'
UNION ALL
SELECT MyFirstColumn, MySecondColumn
FROM   Red_Table
WHERE  @Todays_Color = 'Red';
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

OR Replacement

- An OR statement can impact:
  - Index usage
  - Performance
  - Clarity

```
SELECT MyFirstColumn, MySecondColumn
FROM   Color_Table
WHERE  Color_Choice = 'Black' OR 'Blue';
```

- Instead, use UNION ALL:

```
SELECT MyFirstColumn, MySecondColumn
FROM   Color_Table
WHERE  Color_Choice = 'Black'
UNION ALL
SELECT MyFirstColumn, MySecondColumn
FROM   Color_Table
WHERE  Color_Choice = 'Blue';
```

- Remember: `Column IN (val1, val2, … valn)` ≈ `Column = val1 OR Column = val2 … OR Column = valn`



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Still With Me So Far?



Copyright © 2026, SQLRV, LLC - All Rights Reserved

To Recap…

- Remember the Hidden Assassins:
  - Sergeant Non-SARGable
  - The Data Blob
  - Scorched Cursor
  - The Loopers
  - Sub-Query
  - The Assassin’s UNION








Copyright © 2026, SQLRV, LLC - All Rights Reserved

To Recap…

- Call Upon the Assassin Assassins:
  - The Deliberate Loop
  - The Tally
  - The UNION ALL





Copyright © 2026, SQLRV, LLC - All Rights Reserved

To Recap…

- Examine what may be happening under the hood
  - Consider what an application, API, or user interface may be doing
  - Look for alternatives to the current status quo
  - Work WITH the engine – not against it
- Thinking differently will require different approaches

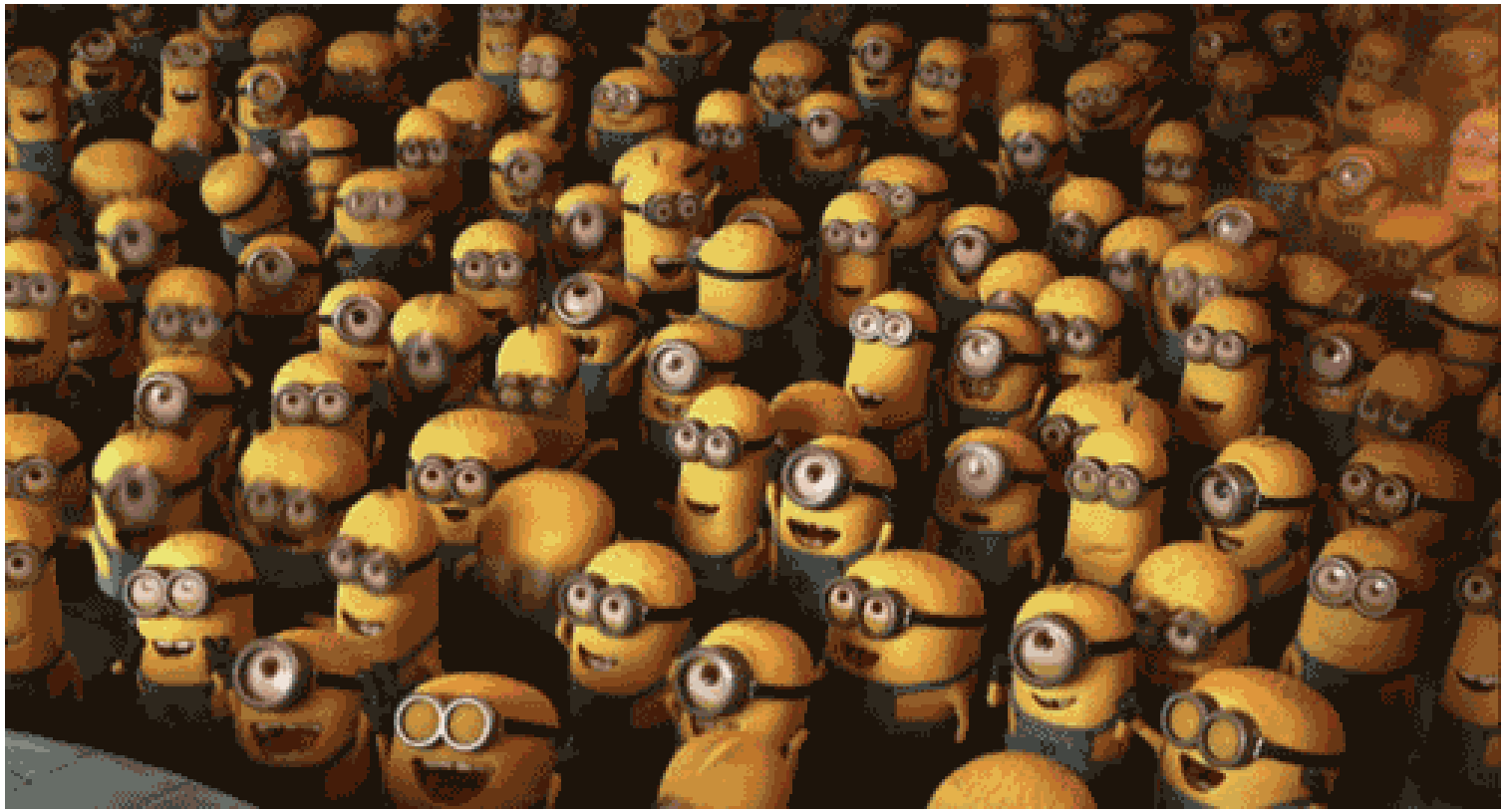


"Unlearn what you have learned, you must!"

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Thanks to our Sponsors!

Without them, we wouldn’t be here!!



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Questions & Comments

BONUS:
A **TON** of free eBooks from RedGate and others (databases and programming)!

PRESENTATION FEEDBACK:
- Your thoughts needed
- Improve presentations
- Make this event even more valuable!!!

Aaron N. Cutshall

www.linkedin.com/in/sqlrv
aaron@sqlrv.com
www.sqlrv.com
www.youtube.com/@sqlrv



Copyright © 2026, SQLRV LLC - All Rights Reserved